Support N-D tensors in Bilinear #5764

New issue

Merged **soumith** merged 5 commits into `pytorch:master` from `li-roy:bilinear` on Mar 17, 2018

support n-d inputs in bilinear and move to aten

Roy Li committed on Mar 14, 2018

commit 1e8917de39d86c2a6d81daa24b67df23d250f637

19 aten/src/ATen/native/Linear.cpp

...	...	@@ -0,0 +1,19 @@
	1	+ #include "ATen/ATen.h"
	2	+ #include "ATen/NativeFunctions.h"
	3	+
	4	+
	5	+ namespace at { namespace native {
	6	+
	7	+ Tensor bilinear(const Tensor& input1, const Tensor& input2, const Tensor& weight, const Tensor& bias) {
	8	+ auto b_input1 = input1.unsqueeze(-2).unsqueeze(-2);

This comment was marked as off-topic. Sign in to view

```
+   auto b_input2 = input2.unsqueeze(-2).unsqueeze(-1);
+
+   auto output = at::matmul(at::matmul(b_input1, weight), b_input2);
+   output = output.squeeze(-1).squeeze(-2).sum(-1);
```

This comment was marked as off-topic. Sign in to view

```
+   if (bias.defined()) {
+     output = output + bias;
+   }
+   return output;
+ }
+
+ }}  // namespace at::native
```

3 aten/src/ATen/native/native_functions.yaml

```
@@ -71,6 +71,9 @@

- func: bernoulli_(Tensor self, double p=0.5, Generator* generator=nullptr) -> Tensor

+ - func: bilinear(Tensor input1, Tensor input2, Tensor weight, Tensor? bias) -> Tensor
+   variants: function
+
- func: cat(TensorList tensors, int64_t dim=0) -> Tensor
  variants: function
```

7 test/test_nn.py

```
@@ -4145,6 +4145,13 @@ def test_bilinear(self):
                _assertGradAndGradgradChecks(self, lambda x1, x2: F.bilinear(x1, x2, module.weight, module.bias),
                                             (input1_1, input2_1))

+    def test_bilinear_broadcasting(self):
```

```
4149 +         m = nn.Bilinear(5, 6, 8)
4150 +         input1 = torch.randn(2, 3, 5)
4151 +         input2 = torch.randn(2, 3, 6)
4152 +         expected = m(input1.view(6, 5), input2.view(6, 6)).view(2, 3, 8)
4153 +         self.assertEqual(expected, m(input1, input2))
4154 +
4148 4155       def test_conv_tbc(self):
4149 4156           inp = Variable(torch.randn(9, 4, 5), requires_grad=True)
4150 4157           weight = Variable(torch.randn(3, 5, 6), requires_grad=True)
```

5 torch/nn/functional.py

```
@@ -1001,10 +1001,7 @@ def linear(input, weight, bias=None):
1001 1001
1002 1002
1003 1003   def bilinear(input1, input2, weight, bias=None):
1004      -     if bias is None:
1005      -         return Bilinear.apply(input1, input2, weight)
1006      -     else:
1007      -         return Bilinear.apply(input1, input2, weight, bias)
     1004 +     return torch._C._VariableFunctions.bilinear(input1, input2, weight, bias)
1008 1005
1009 1006
1010 1007   def embedding(input, weight, padding_idx=None, max_norm=None, norm_type=2,
```

5 torch/nn/modules/linear.py

```
@@ -74,7 +74,10 @@ class Bilinear(Module):
74 74
75 75       Shape:
76 76           - Input: :math:`(N, \text{in1_features})`, :math:`(N, \text{in2_features})`
77      -         - Output: :math:`(N, \text{out_features})`
     77 +           where :math:`*` means any number of additional dimensions. All but the last
```

Sign in to view

This comment was marked as off-topic.

```
+           dimension of the inputs should be the same.
+         - Output: :math:`(N, \text{out_features})` where all but the last dimension
+           are the same shape as the input.

      Attributes:
          weight: the learnable weights of the module of shape
```

Terms Privacy Security Status Docs Contact Manage cookies
Do not share my personal information
© 2025 GitHub, Inc.

Commit 1dcad08



li-roy authored and soumith committed on Mar 17, 2018

```
Support N-D tensors in Bilinear (#5764)
* support n-d inputs in bilinear and move to aten
* support n-d inputs in bilinear and move to aten
* add asserts to bilinear inputs
* address comments
* cast int64_t in asserts
```

main (#5764) · whc_flight_4 ··· bc2caa7fdf006894eff7af936babde69ab5a40f8-huydhn-debug

1 parent 04edb89 commit 1dcad08

6 files changed +52 -65 lines changed

Search within code

aten/src/ATen/native/Linear.cpp +36

```
@@ -0,0 +1,36 @@
1  + #include "ATen/ATen.h"
2  + #include "ATen/NativeFunctions.h"
3  +
4  +
5  + namespace at { namespace native {
6  +
7  + Tensor bilinear(const Tensor& input1, const Tensor& input2, const Tensor&
     weight, const Tensor& bias) {
8  +   AT_ASSERT(input1.dim() == input2.dim(), "bilinear(): input dimensions do
     not match: got %lld and %lld",
9  +             (long long)input1.dim(), (long long)input2.dim());
10 +   for (int64_t i = 0; i < input1.dim() - 1; i++) {
```

```
+    AT_ASSERT(input1.size(i) == input2.size(i),
+              "bilinear(): input batch dimensions do not match at dim %lld: got %lld and %lld",
+              (long long)i, (long long)input1.size(i), (long long)input2.size(i));
+  }
+  AT_ASSERT(input1.size(input1.dim() - 1) == weight.size(1),
+            "bilinear(): input1 size does not match weight size: got %lld but expected %lld",
+            (long long)input1.size(input1.dim() - 1), (long long)weight.size(1));
+  AT_ASSERT(input2.size(input2.dim() - 1) == weight.size(2),
+            "bilinear(): input2 size does not match weight size: got %lld but expected %lld",
+            (long long)input2.size(input2.dim() - 1), (long long)weight.size(2));
+  AT_ASSERT(bias.defined() && bias.size(0) == weight.size(0),
+            "bilinear(): bias size does not match weight size: got %lld but expected %lld",
+            (long long)bias.size(0), (long long)weight.size(0));
+
+  auto b_input1 = input1.unsqueeze(-2).unsqueeze(-2);
+  auto b_input2 = input2.unsqueeze(-2).unsqueeze(-1);
+
+  auto output = at::matmul(at::matmul(b_input1, weight), b_input2);
+  output = output.squeeze(-1).squeeze(-1);
+  if (bias.defined()) {
+    output = output + bias;
+  }
+  return output;
+}
+
+}}  // namespace at::native
```

aten/src/ATen/native/native_functions.yaml +3

```
@@ -71,6 +71,9 @@

- func: bernoulli_(Tensor self, double p=0.5, Generator* generator=nullptr) -> Tensor
```

```
73  73
    74  + - func: bilinear(Tensor input1, Tensor input2, Tensor weight, Tensor?
          bias) -> Tensor
    75  +   variants: function
    76  +
74  77    - func: cat(TensorList tensors, int64_t dim=0) -> Tensor
75  78      variants: function
76  79
```

test/test_nn.py +7

```
                @@ -4169,6 +4169,13 @@ def test_bilinear(self):
4169  4169              _assertGradAndGradgradChecks(self, lambda x1, x2:
                      F.bilinear(x1, x2, module.weight, module.bias),
4170  4170                                           (input1_1, input2_1))
4171  4171
    4172  +     def test_bilinear_broadcasting(self):
    4173  +         m = nn.Bilinear(5, 6, 8)
    4174  +         input1 = torch.randn(2, 3, 5)
    4175  +         input2 = torch.randn(2, 3, 6)
    4176  +         expected = m(input1.view(6, 5), input2.view(6, 6)).view(2, 3,
              8)
    4177  +         self.assertEqual(expected, m(input1, input2))
    4178  +
4172  4179      def test_conv_tbc(self):
4173  4180          inp = Variable(torch.randn(9, 4, 5), requires_grad=True)
4174  4181          weight = Variable(torch.randn(3, 5, 6), requires_grad=True)
```

torch/nn/_functions/linear.py -58

Load Diff

This file was deleted.

torch/nn/functional.py +1 -5

```
@@ -9,7 +9,6 @@
 from torch._C import _infer_size, _add_docstr
 from . import _functions
 from .modules import utils
-from ._functions.linear import Bilinear
 from ._functions.padding import ConstantPadNd
 from ._functions import vision
 from ._functions.thnn.fold import Col2Im, Im2Col
@@ -1001,10 +1000,7 @@ def linear(input, weight, bias=None):


 def bilinear(input1, input2, weight, bias=None):
-    if bias is None:
-        return Bilinear.apply(input1, input2, weight)
-    else:
-        return Bilinear.apply(input1, input2, weight, bias)
+    return torch._C._VariableFunctions.bilinear(input1, input2, weight, bias)


 def embedding(input, weight, padding_idx=None, max_norm=None, norm_type=2,
```

torch/nn/modules/linear.py +5 -2

```
@@ -73,8 +73,11 @@ class Bilinear(Module):
                 Default: ``True``

     Shape:
-        - Input: :math:`(N, \text{in1_features})`, :math:`(N, \text{in2_features})`
-        - Output: :math:`(N, \text{out_features})`
+        - Input: :math:`(N, *, \text{in1_features})`, :math:`(N, *, \text{in2_features})`
+          where :math:`*` means any number of additional dimensions. All but the last
+          dimension of the inputs should be the same.
+
+        - Output: :math:`(N, *, \text{out_features})`

     Attributes:
         weight: the learnable weights of the module of shape
```

```
            - Output: :math:`(N, *, \text{out_features})` where all but the
  last dimension
+             are the same shape as the input.

        Attributes:
            weight: the learnable weights of the module of shape
```

Comments 0 (0)

Please sign in to comment.